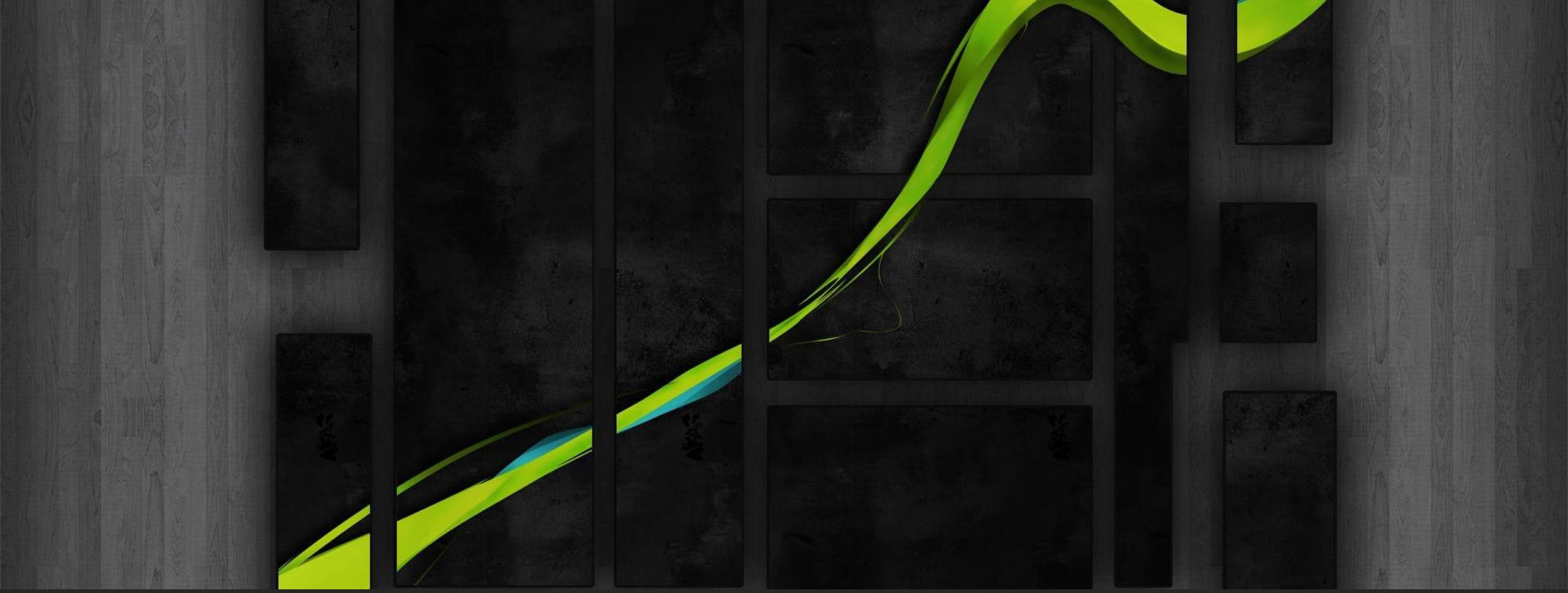
Memory

2023/12/11

B113040045 許育菖



Exercise 1: 4 x 4 RAM

題目說明

- Design and verify the 4 x 4 RAM using Verilog HDL.
  - Structural modeling

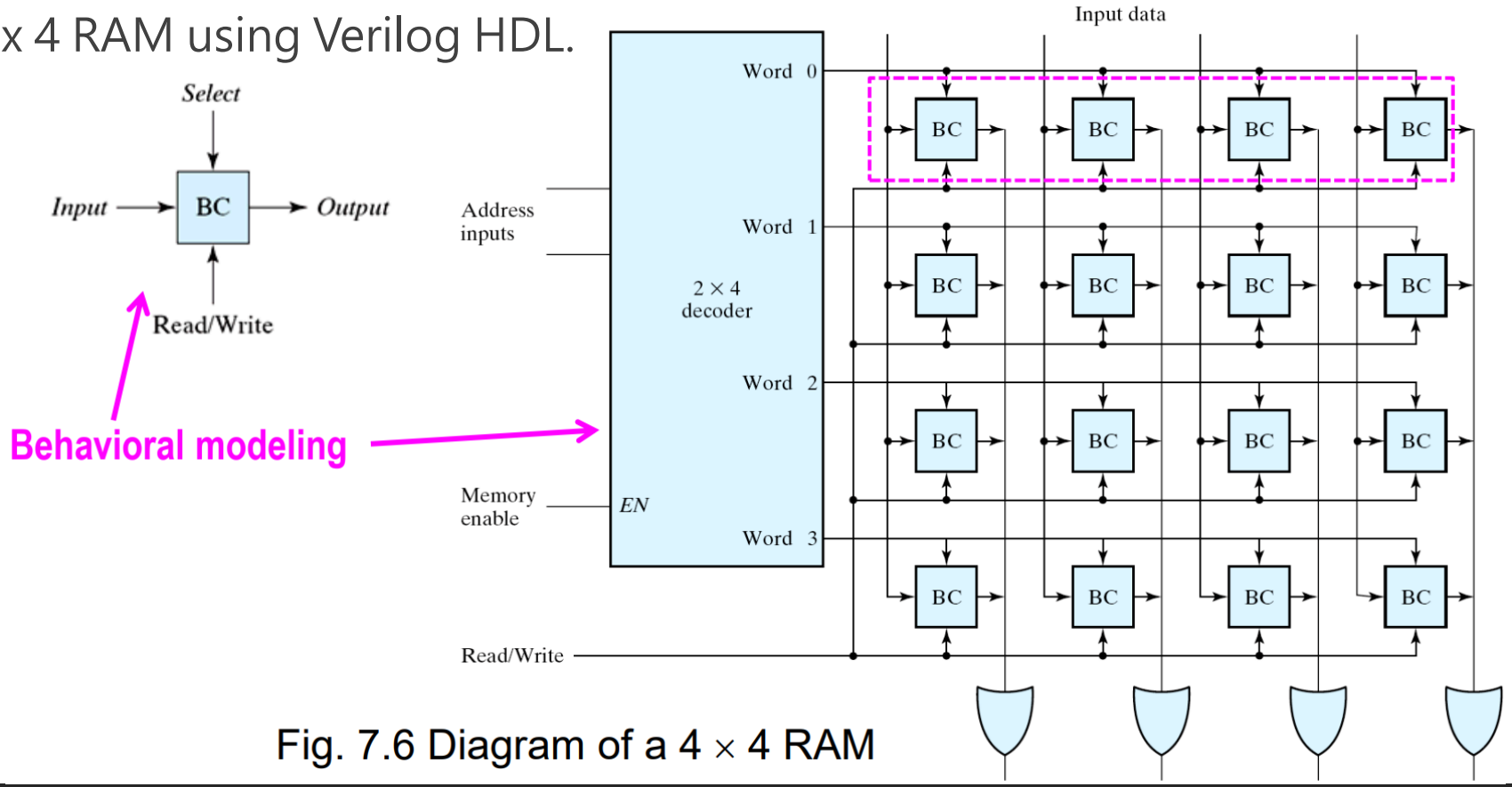


Fig. 7.6 Diagram of a 4 × 4 RAM

設計邏輯 1

在Fig 7.6中，我們可以發現在製作4×4 RAM時，需要使用到2×4 Decoder以及BC。因此我們可以先將這兩者的電路設計出來， 2×4 Decoder 之前學過了；而BC的電路圖如Fig 7.5所示。

注意，雖然Fig 7.5有將BC的電路圖畫出來，但建議還是使用題目推薦的**Behavioral modeling** 來撰寫程式碼，不要使用 **Structural modeling** ，我一開始使用**Structural modeling**無法將正確的波型圖呈現出來。

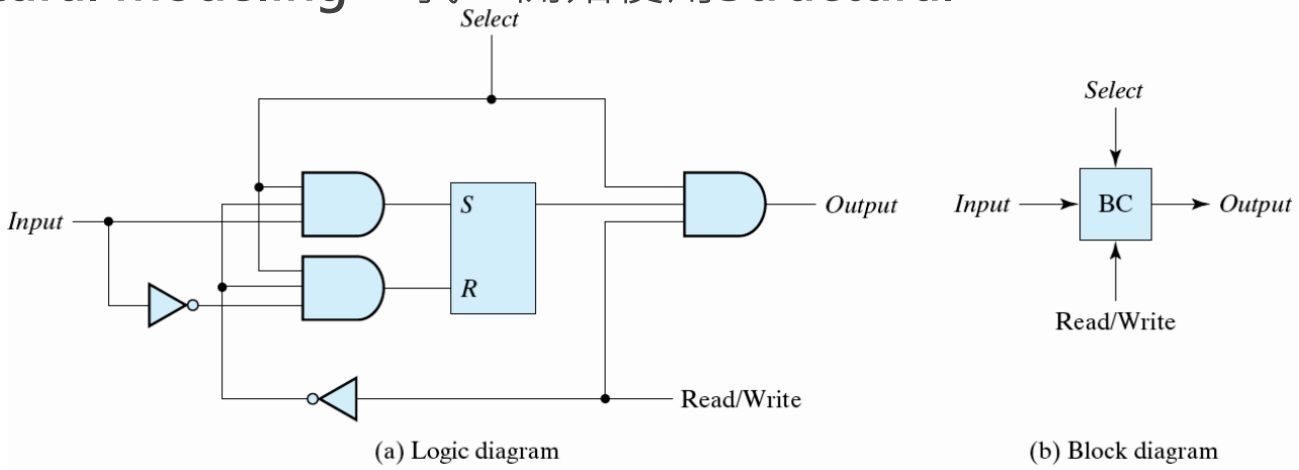


Fig. 7.5 Memory cell

Decoder

```
•timescale 1ns / 1ps
•module decoder(
•    output a, b, c, d,
•    input in1, in2,
•    input enable );
•
•    reg [3:0] out;
•always @ (enable or in1 or in2)
•    begin
•        out = 0;
•        if(enable) begin
•            case(in1 * 2 + in2)
•                2'b00: out = 4'b0001;
•                2'b01: out = 4'b0010;
•                2'b10: out = 4'b0100;
•                2'b11: out = 4'b1000;
•            endcase
•        end
•    end
•    assign a = out[0];
•    assign b = out[1];
•    assign c = out[2];
•    assign d = out[3];
• endmodule
```

BC

```
`timescale 1ns / 1ps
module BC(
    output reg out,
    input in, select, ReadWrite);

    reg mem;

    always @ (select or ReadWrite or in)
        if(select)
            if(ReadWrite)
                out = mem;
            else
                mem = in;
        else
            out = 1'b0;
endmodule
```

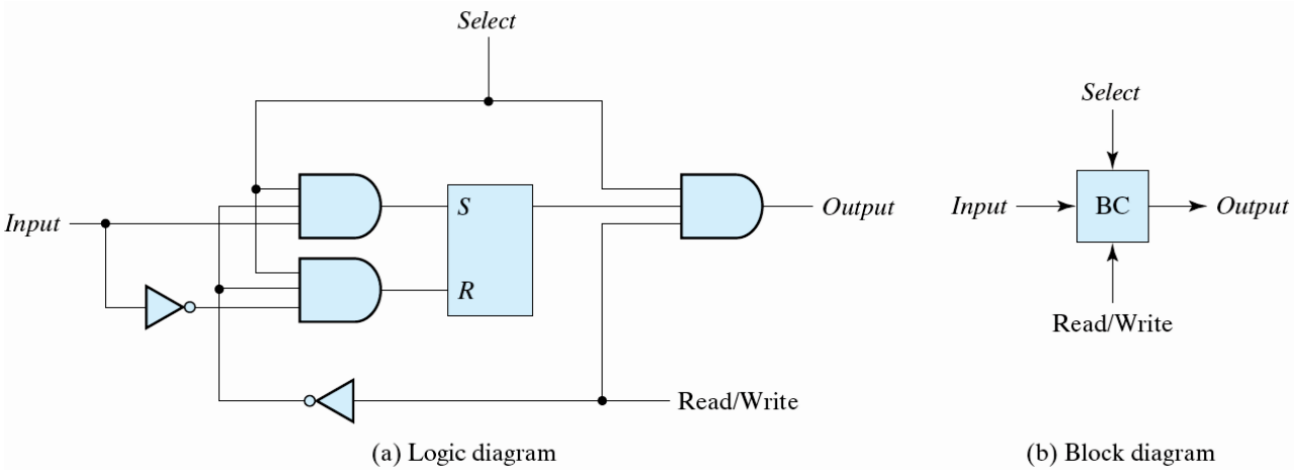
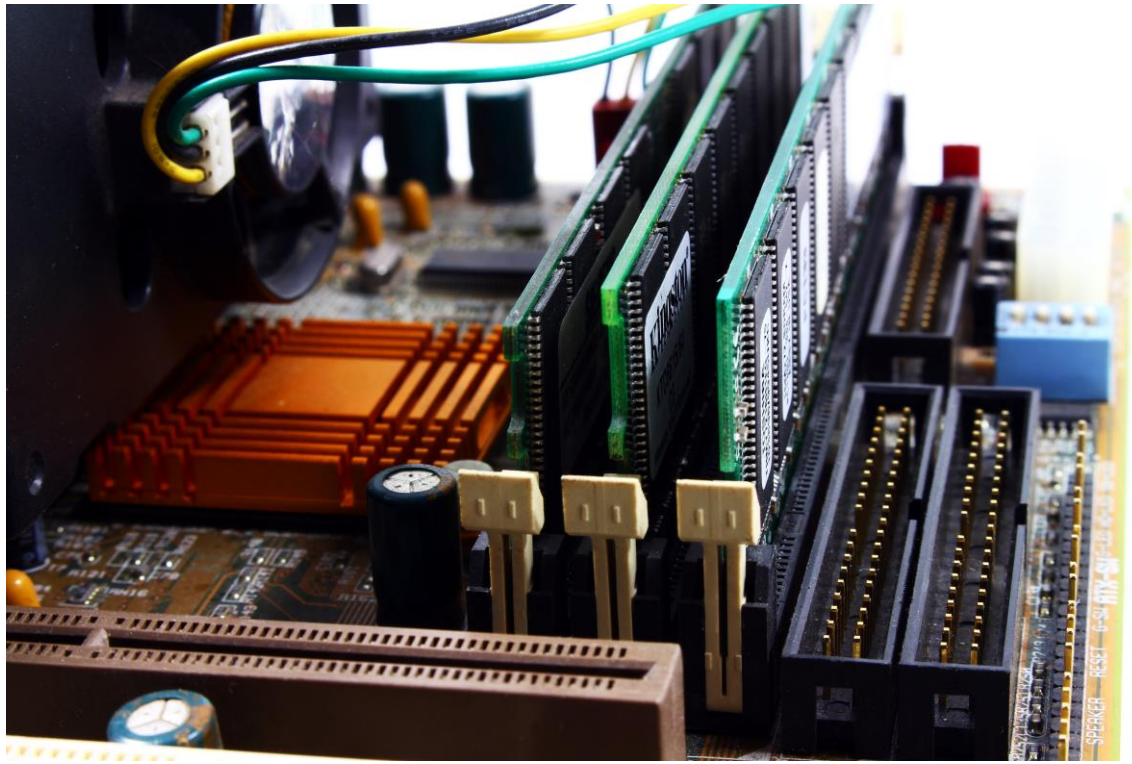


Fig. 7.5 Memory cell

設計邏輯 2

設計好Decoder和BC後，我們就可以按照Fig 7.6的電路，將RAM「組裝」起來了。



4×4 RAM

```
`timescale 1ns / 1ps
module exercise1(
    output out0, out1, out2, out3,
    input addressA, addressB, enable, ReadWrite, d0, d1, d2, d3);

    wire w0, w1, w2, w3;
    decoder DC(w0, w1, w2, w3, addressA, addressB, enable);
    wire n00, n01, n02, n03;
    BC bc00(n00, d0, w0, ReadWrite);
    BC bc01(n01, d0, w1, ReadWrite);
    BC bc02(n02, d0, w2, ReadWrite);
    BC bc03(n03, d0, w3, ReadWrite);
    or(out0, n00, n01, n02, n03);
    wire n10, n11, n12, n13;
    BC bc10(n10, d1, w0, ReadWrite);
    BC bc11(n11, d1, w1, ReadWrite);
    BC bc12(n12, d1, w2, ReadWrite);
    BC bc13(n13, d1, w3, ReadWrite);
    or(out1, n10, n11, n12, n13);

    wire n20, n21, n22, n23;
    BC bc20(n20, d2, w0, ReadWrite);
    BC bc21(n21, d2, w1, ReadWrite);
    BC bc22(n22, d2, w2, ReadWrite);
    BC bc23(n23, d2, w3, ReadWrite);
    or(out2, n20, n21, n22, n23);
    wire n30, n31, n32, n33;
    BC bc30(n30, d3, w0, ReadWrite);
    BC bc31(n31, d3, w1, ReadWrite);
    BC bc32(n32, d3, w2, ReadWrite);
    BC bc33(n33, d3, w3, ReadWrite);
    or(out3, n30, n31, n32, n33);

endmodule
```

Testbench

以ReadWrite來控制讀檔和寫檔

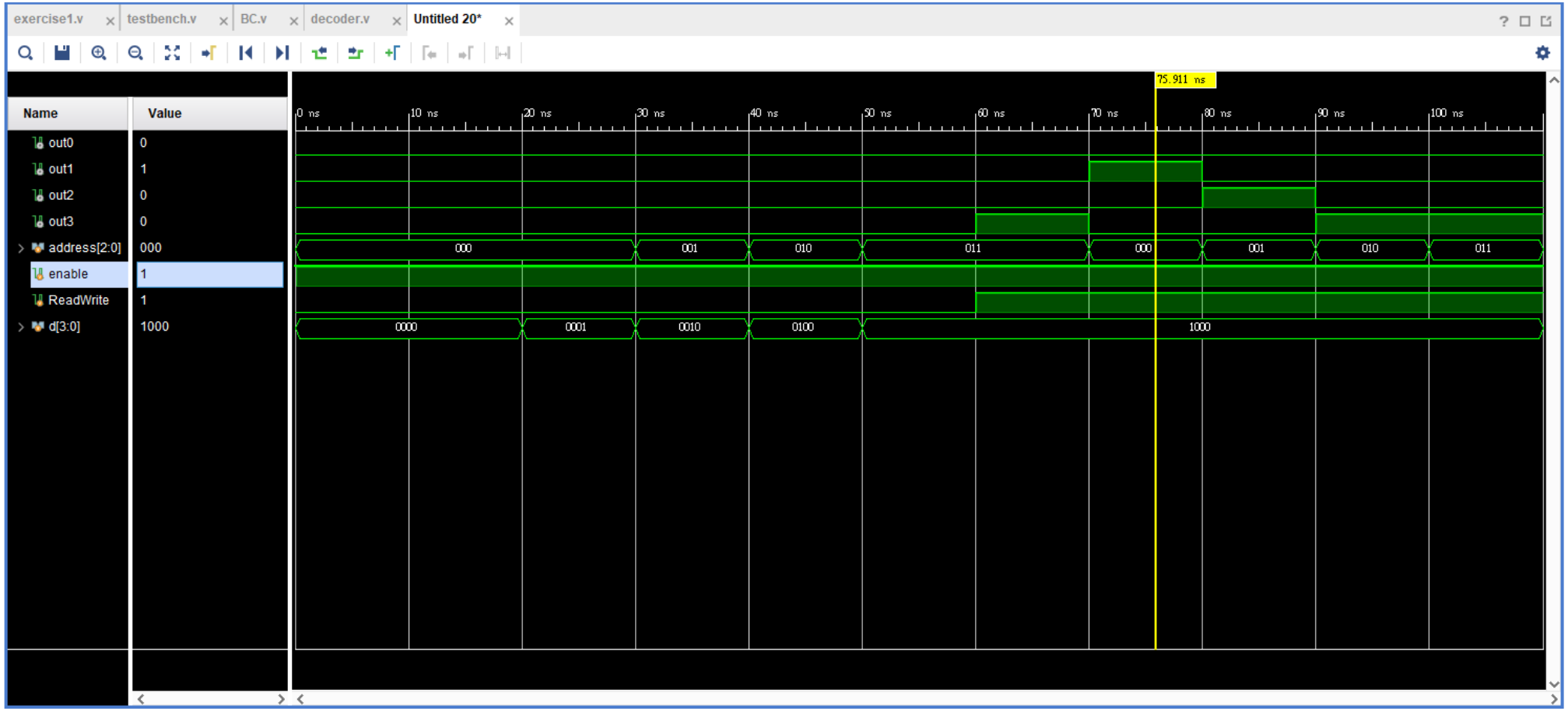
```
`timescale 1ns / 1ps
module testbench();
    wire out0, out1, out2, out3;
    reg [2:0] address;
    reg enable, ReadWrite;
    reg [3:0] d;

    exercise1 M0 (out0, out1, out2, out3, address[0], address[1], enable, ReadWrite, d[0], d[1], d[2], d[3]);
```

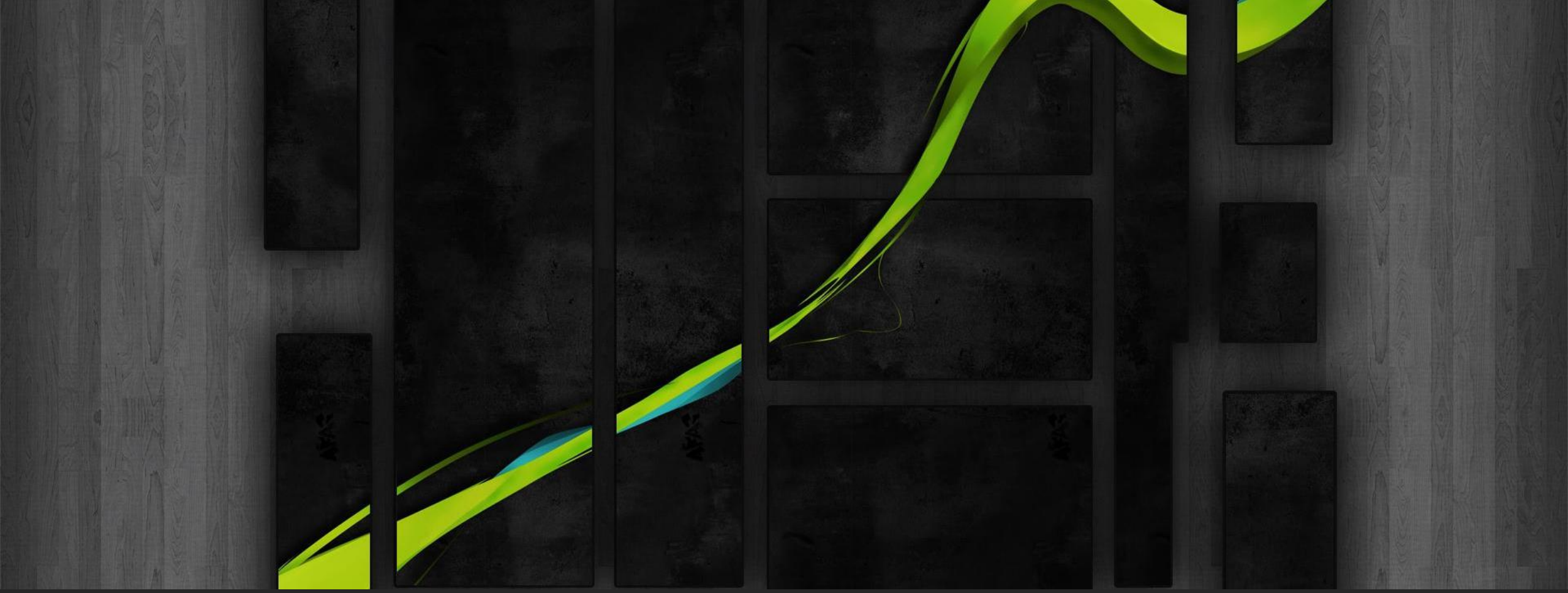
```
initial #110 $finish;
  initial begin
      enable = 1;
      ReadWrite = 0;
      address = 0;
      d = 0;

      #10 ReadWrite = 1'b0; //Write
      #10 address = 2'b00; d = 4'b0001;
      #10 address = 2'b01; d = 4'b0010;
      #10 address = 2'b10; d = 4'b0100;
      #10 address = 2'b11; d = 4'b1000;
```

```
#10 ReadWrite = 1'b1; //Read
      #10 address = 2'b00;
      #10 address = 2'b01;
      #10 address = 2'b10;
      #10 address = 2'b11;
      #10;
  end
endmodule
```

實驗結果

波型圖全部往後了一格

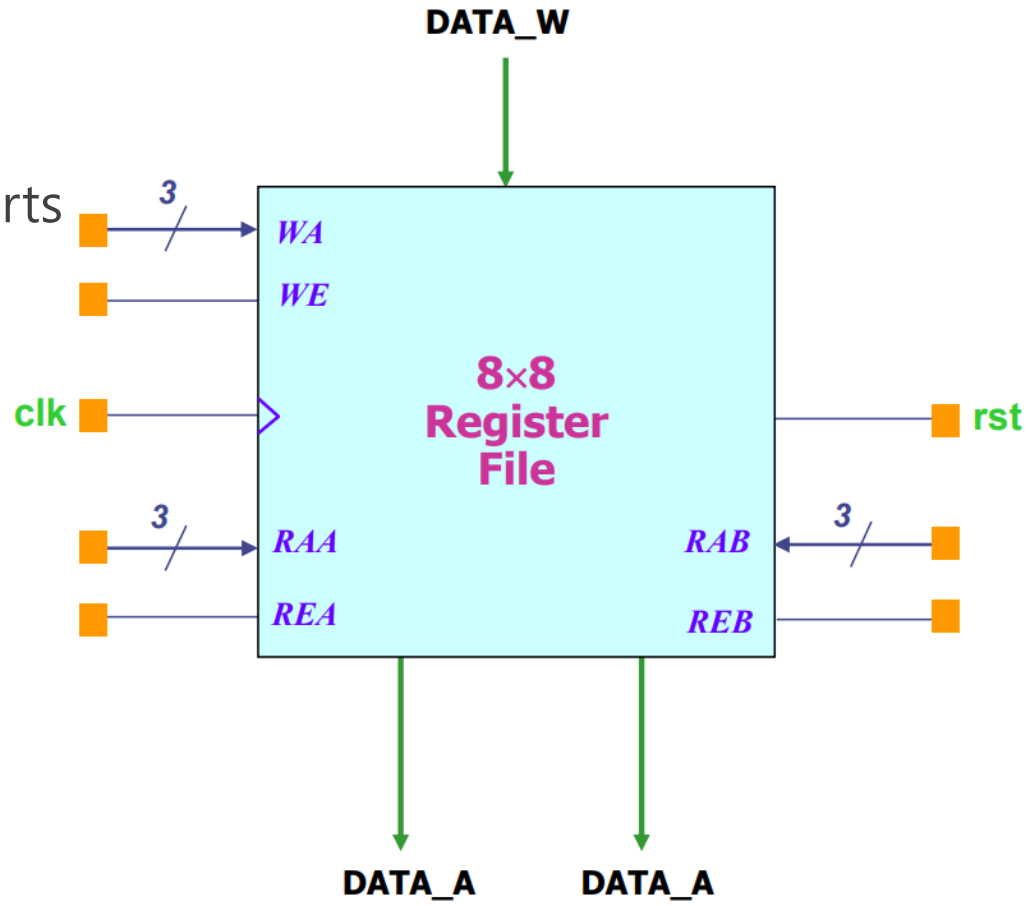




Exercise 2: 8 x 8 Register File

題目說明

- Register file is used as fast temporary storage
- Design and verify the 8 x 8 register file with 2 read ports and 1 write port using Verilog HDL
  - **Behavioral modeling**



8 x 8 register

```
`timescale 1ns / 1ps

module exercise2(WA, WE, RAA, REA, RAB, REB,
DATA_W, DATA_A, DATA_B, clk, rst);
    input [2:0] WA, RAA, RAB;
    input WE, REA, REB, clk, rst;
    input [7:0] DATA_W;
    output [7:0] DATA_A, DATA_B;

    reg [7:0] registers[7:0];

    assign DATA_A = registers[RAA];
    assign DATA_B = registers[RAB];
```

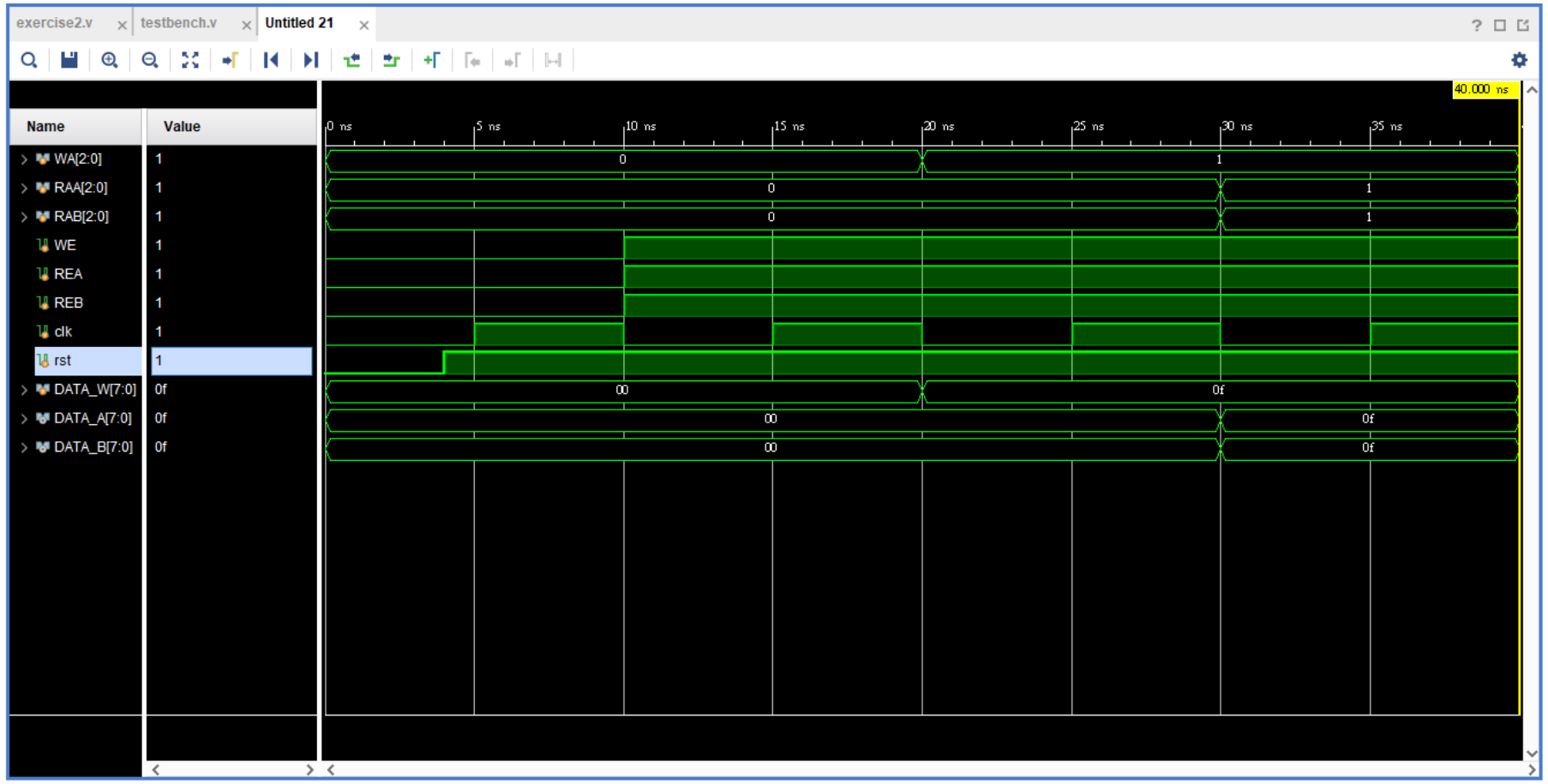
```
always @ (posedge clk or negedge rst)
begin
    if (~rst)
        registers[0] = 0;
    else
    begin
        if (WE) registers[WA] <= DATA_W;
    end
end
endmodule
```

Testbanch

檢測暫存器是否可以讀入和寫出

```
`timescale 1ns / 1ps
module testbench();
   reg [2:0] WA, RAA, RAB;
   reg WE, REA, REB, clk, rst;
   reg [7:0] DATA_W;
   wire [7:0] DATA_A, DATA_B;

   exercise2 U2(WA, WE, RAA, REA, RAB, REB, DATA_W, DATA_A, DATA_B, clk, rst);

   initial #150 $finish;
   initial begin clk = 0; forever #5 clk = ~clk; end
   initial begin rst = 0; #4 rst = 1; end
initial begin
      REA = 0;  REB = 0;
      WE = 0;
      WA = 0;
      RAA = 0;
      RAB = 0;
      DATA_W = 0;

      #10
      REA = 1’b1;  REB = 1'b1;
      WE = 1'b1;
      WA = 3'b000;
      RAA = 3'b000;
      RAB = 3'b000;
      DATA_W = 8'b00000000;
      #10
      REA = 1’b1;  REB = 1'b1;
      WE = 1'b1;
      WA = 3'b001;
      RAA = 3'b000;
      RAB = 3'b000;
      DATA_W = 8'b00001111;

      #10
      RAA = 3'b001;
      RAB = 3'b001;
      #10
   $finish;
end
endmodule
```

實驗結果


exercise2.v
testbench.v
Untitled 21
Name
Value
WA[2:0]
RAA[2:0]
RAB[2:0]
WE
REA
REB
clk
rst
DATA_W[7:0]
DATA_A[7:0]
DATA_B[7:0]
0 ns
5 ns
10 ns
15 ns
20 ns
25 ns
30 ns
35 ns
40.000 ns

實驗心得

我覺得這次的實驗非常的困難，不是像以往難在不知道怎麼寫，而是知道怎麼寫，並且成功寫出來了，在邏輯上也正確，但波型圖跑出來卻是錯的==。

特別是第一題的部分，一開始我是以structural modeling的方式來設計BC的電路，但執行發現竟然無法跑出波型圖，輸出的出現紅字，於是才重新以Behavioral modeling的方式再寫一次。結果卻變成波型圖的值全部往後順延一格，真的超無言，我和朋友們檢查很多次都沒發現哪個地方有錯誤，我只能說，感謝助教通融讓我過這一題，不然我大概要在教室留到中午12點了QQ。